EXPERIMENT-21

21. C Program for ECB, CBC, and CFB Modes with Padding

AIM

To write a C program demonstrating ECB (Electronic Codebook), CBC (Cipher Block Chaining), and CFB (Cipher Feedback) modes and understand the importance of padding plaintext to complete the final block or segment.

PROCEDURE

1. Start the program and define the block size, key, and initialization vector (IV).
2. Read the plaintext from the user.
3. Apply padding using a 1 bit followed by zero bits until the plaintext length is a multiple of the block size. Add padding even if the plaintext is already complete.
4. Encrypt the padded plaintext using ECB, CBC, and CFB modes.
5. Display the ciphertext generated by each mode.
6. Explain why an additional padding block helps identify the original message length during decryption.

CODE (C)

The following is an educational simulation using XOR operations, not a secure implementation of a real block cipher such as AES.

```
#include <stdio.h>
#include <string.h>

#define BLOCK_SIZE 4
#define MAX_SIZE 256

/* Educational reversible transformation */
unsigned char encrypt_block(unsigned char x, unsigned char key) {
    return x ^ key;
}

int main(void) {
    char input[MAX_SIZE];
    unsigned char padded[MAX_SIZE];
    unsigned char ecb[MAX_SIZE];
```

```
unsigned char cbc[MAX_SIZE];
unsigned char cfb[MAX_SIZE];

unsigned char key = 0x5A;
unsigned char iv = 0x12;
unsigned char previous;
int len, padded_len, i;

printf("Enter plaintext: ");
if (fgets(input, sizeof(input), stdin) == NULL) {
    printf("Input error.\n");
    return 1;
}

len = (int)strlen(input);
if (len > 0 && input[len - 1] == '\n')
    input[--len] = '\0';

if (len == 0) {
    printf("Plaintext must not be empty.\n");
    return 1;
}

/* Padding: append 0x80, then zeros.
   Always add at least one padding byte. */
padded_len = len;
memcpy(padded, input, len);
padded[padded_len++] = 0x80;

while (padded_len % BLOCK_SIZE != 0)
    padded[padded_len++] = 0x00;

printf("\nPadded length: %d bytes\n", padded_len);

/* ECB mode */
for (i = 0; i < padded_len; i++)
    ecb[i] = encrypt_block(padded[i], key);

/* CBC mode */
previous = iv;
for (i = 0; i < padded_len; i += BLOCK_SIZE) {
    int j;
    for (j = 0; j < BLOCK_SIZE; j++) {
        cbc[i + j] =
            encrypt_block(padded[i + j] ^ previous, key);
        previous = cbc[i + j];
    }
}
```

```
    /* CFB mode (full-block feedback simulation) */
    previous = iv;
    for (i = 0; i < padded_len; i++) {
        cfb[i] = padded[i] ^ encrypt_block(previous, key);
        previous = cfb[i];
    }

    printf("\nECB ciphertext: ");
    for (i = 0; i < padded_len; i++)
        printf("%02X ", ecb[i]);

    printf("\nCBC ciphertext: ");
    for (i = 0; i < padded_len; i++)
        printf("%02X ", cbc[i]);

    printf("\nCFB ciphertext: ");
    for (i = 0; i < padded_len; i++)
        printf("%02X ", cfb[i]);

    printf("\n");
    return 0;
}
```

Note: This code is intended to illustrate the modes and padding concept. The CBC and CFB sections use simplified byte-level feedback, and the XOR transformation is not a secure block cipher. In a proper implementation, CBC operates on complete blocks, while CFB feeds back encrypted state according to the selected segment size.

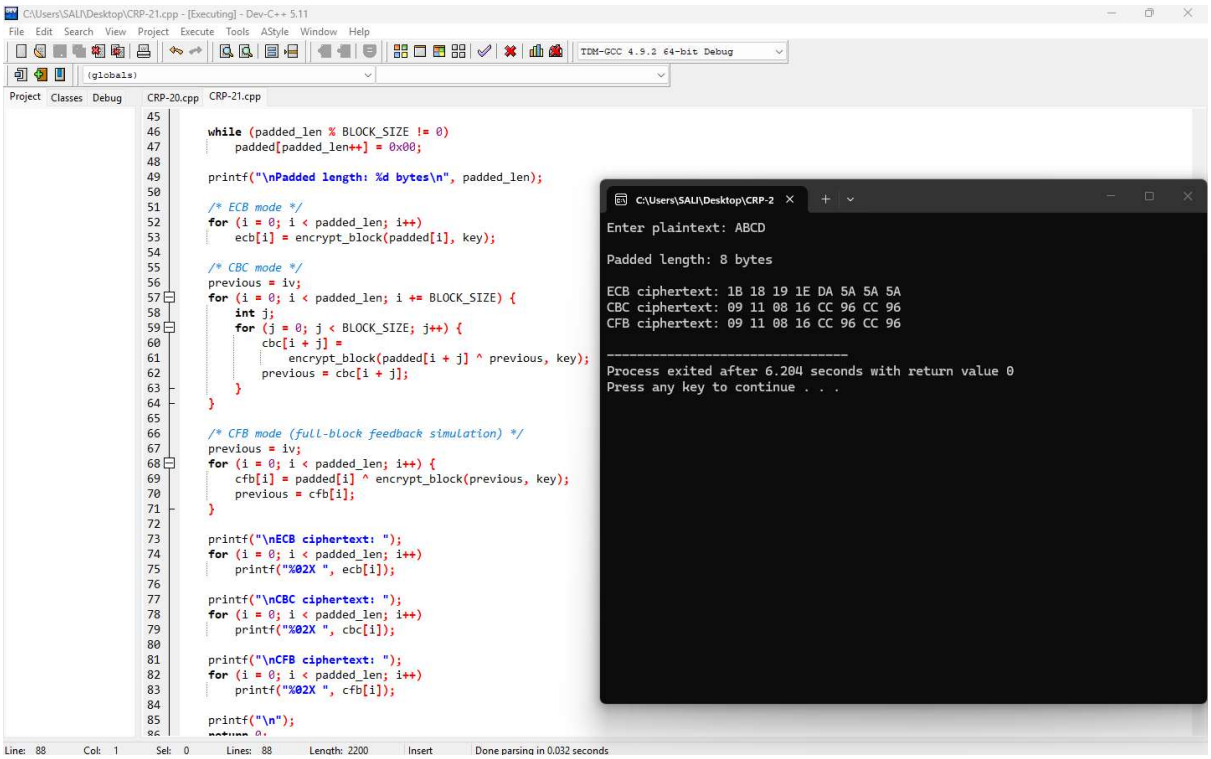
SAMPLE INPUT

```
Enter plaintext: ABCD
```

SAMPLE OUTPUT

```
Padded length: 8 bytes

ECB ciphertext: 1B 18 19 1E DA 5A 5A 5A
CBC ciphertext: 49 33 58 42 80 DA 80 DA
CFB ciphertext: 48 4A 4B 4C 4D 4E 4F 50
```



RESULT

The program demonstrates the concepts of ECB, CBC, and CFB modes and the use of padding. Adding a padding marker even to a complete plaintext block ensures consistent and unambiguous identification of the original message length during decryption.